Longest Common Subsequence (LCS)

Last Updated : 23 Jul, 2025

Given two strings, **s1** and **s2**, the task is to find the length of the Longest Common Subsequence. If there is no **common subsequence**, return `0`. A **subsequence** is a string generated from the original string by deleting 0 or more characters, without changing the relative order of the remaining characters.

For example, subsequences of "ABC" are "", "A", "B", "C", "AB", "AC", "BC" and "ABC". In general, a string of length **n** has 2^n subsequences.

Examples:

Input: *s1 = "ABC", s2 = "ACD"*
Output: *2*
Explanation: *The longest subsequence which is present in both strings is "AC".*

Input: *s1 = "AGGTAB", s2 = "GXTXAYB"*
Output: *4*
Explanation: *The longest common subsequence is "GTAB".*

Input: *s1 = "ABC", s2 = "CBA"*
Output: *1*
Explanation: *There are three longest common subsequences of length 1, "A", "B" and "C".*

Table of Content

[Naive Approach] Using Recursion - O(2 ^ min(m, n)) Time and O(min(m, n)) Space

*The idea is to compare the last characters of **s1** and **s2**. While comparing the strings **s1** and **s2** two cases arise:*

1. ***Match** : Make the recursion call for the remaining strings (strings of lengths **m-1** and **n-1**) and add 1 to result.*
2. ***Do not Match** : Make two recursive calls. First for lengths **m-1** and **n**, and second for **m** and **n-1.** Take the maximum of two results.*

***Base case** : If any of the strings become empty, we return 0.*

For example, consider the input strings s1 = "ABX" and s2 = "ACX".

LCS("ABX", "ACX") = 1 + LCS("AB", "AC") [Last Characters Match]

LCS("AB", "AC") = max(LCS("A", "AC") , LCS("AB", "A")) [Last Characters Do Not Match]

LCS("A", "AC") = max(LCS("", "AC") , LCS("A", "A")) = max(0, 1 + LCS("", "")) = 1

LCS("AB", "A") = max(LCS("A", "A") , LCS("AB", "")) = max(1 + LCS("", "", 0)) = 1

*So overall result is 1 + 1 = **2***

C++ C Java Python C# JavaScript

```cpp
// A Naive recursive implementation of LCS problem
#include <bits/stdc++.h>
using namespace std;

// Returns length of LCS for s1[0..m-1], s2[0..n-1]
int lcsRec(string &s1, string &s2,int m,int n) {

    // Base case: If either string is empty, the length of LCS is
0
    if (m == 0 || n == 0)
        return 0;
```

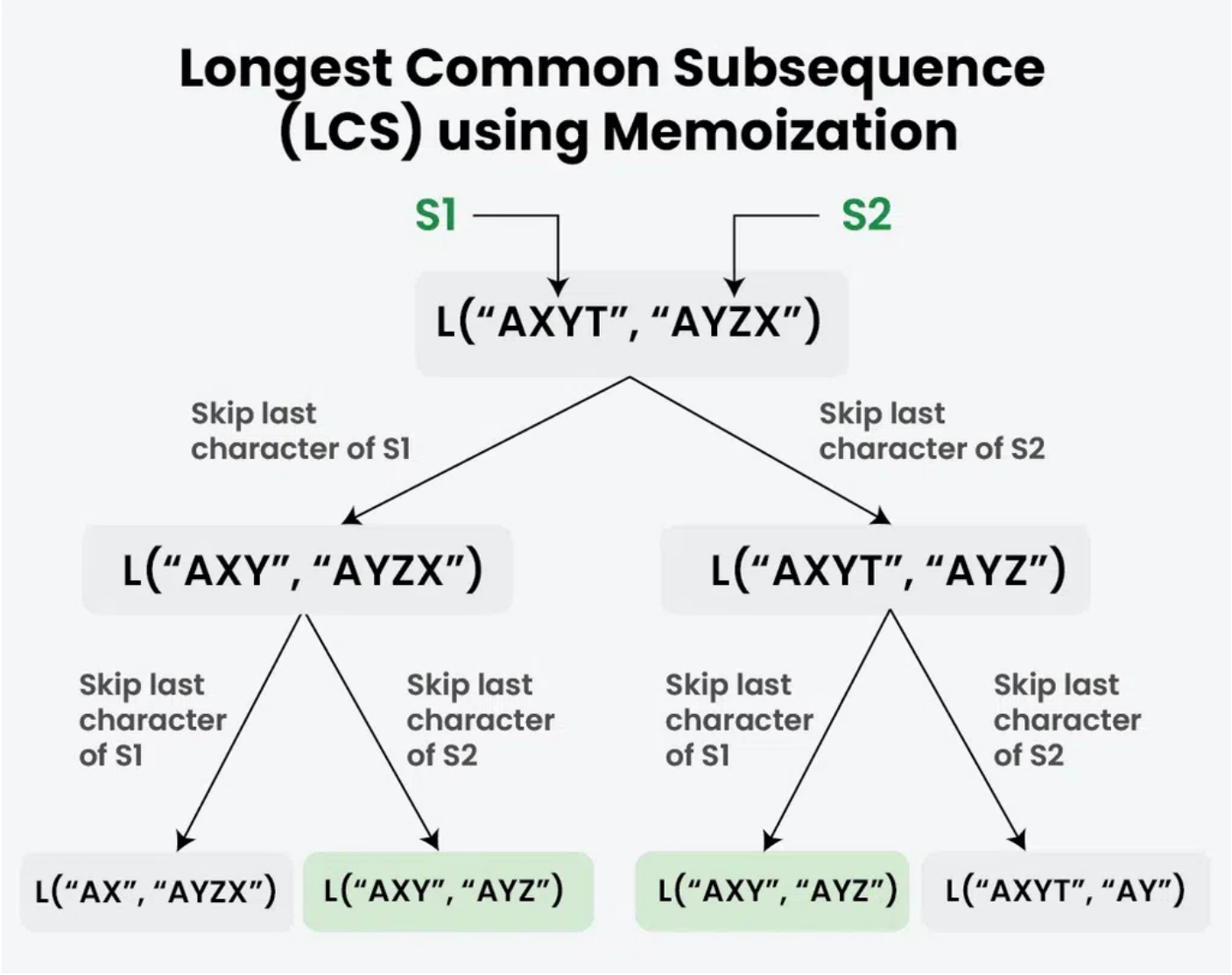
```
    // If the last characters of both substrings match
    if (s1[m - 1] == s2[n - 1])

        // Include this character in LCS and recur for remaining
substrings
        return 1 + lcsRec(s1, s2, m - 1, n - 1);

    else
        // If the last characters do not match
        // Recur for two cases:
        // 1. Exclude the last character of s1
        // 2. Exclude the last character of s2
        // Take the maximum of these two recursive calls
        return max(lcsRec(s1, s2, m, n - 1), lcsRec(s1, s2, m - 1,
n));
}
int lcs(string &s1,string &s2){

    int m = s1.size(), n = s2.size();
    return lcsRec(s1,s2,m,n);
}

int main() {
    string s1 = "AGGTAB";
    string s2 = "GXTXAYB";
    int m = s1.size();
    int n = s2.size();

    cout << lcs(s1, s2) << endl;

    return 0;
}
```

Output

```
4
```

[Better Approach] Using Memoization (Top Down DP) - O(m * n) Time and O(m * n) Space

*To optimize the recursive solution, we use a **2D memoization table** of size (m+1)×(n+1)(m+1) \times (n+1)(m+1)×(n+1), initialized to −1-1−1 to track computed values. Before making recursive calls, we check this table to avoid redundant computations of overlapping subproblems. This prevents repeated calculations, improving efficiency through **memoization or tabulation**.*



Overlapping Subproblems in Longest Common Subsequence

C++ C Java Python C# JavaScript

```cpp
// C++ implementation of Top-Down DP
// of LCS problem
#include <bits/stdc++.h>
using namespace std;
// Returns length of LCS for s1[0..m-1], s2[0..n-1]
int lcsRec(string &s1, string &s2, int m, int n,
vector<vector<int>> &memo) {

    // Base Case
```

```
    if (m == 0 || n == 0)
        return 0;

    // Already exists in the memo table
    if (memo[m][n] != -1)
        return memo[m][n];

    // Match
    if (s1[m - 1] == s2[n - 1])
        return memo[m][n] = 1 + lcsRec(s1, s2, m - 1, n - 1,
memo);

    // Do not match
    return memo[m][n] = max(lcsRec(s1, s2, m, n - 1, memo),
lcsRec(s1, s2, m - 1, n, memo));
}
int lcs(string &s1,string &s2){
    int m = s1.length();
    int n = s2.length();
    vector<vector<int>> memo(m + 1, vector<int>(n + 1, -1));
    return lcsRec(s1, s2, m, n, memo);
}

int main() {
    string s1 = "AGGTAB";
    string s2 = "GXTXAYB";
    cout << lcs(s1, s2) << endl;
    return 0;
}
```

Output

```
4
```

[Expected Approach 1] Using Bottom-Up DP (Tabulation) - O(m * n) Time and O(m * n) Space

There are two parameters that change in the recursive solution and these parameters go from 0 to m and 0 to n. So we create a 2D dp array of size (m+1) x (n+1).

- *We first fill the known entries when m is 0 or n is 0.*
- *Then we fill the remaining entries using the recursive formula.*

Say the strings are **S1 = "AXTY"** and **S2 = "AYZX"**, Follow below :

Initially | Initially create a 2D matrix (say dp[][]) of size 5x5 whose first row and first column are filled with 0.

		A	Y	Z	X
	0	1	2	3	4
0	0	0	0	0	0
A 1	0				
X 2	0				
Y 3	0				
T 4	0				

01 Step | Now, for dp[1][1] , dp[1][1] = 1 + dp[0][0] as the characters are equal.

		A	Y	Z	X
	0	1	2	3	4
0	0	0	0	0	0
A 1	0	1	1	1	1
X 2	0				
Y 3	0				
T 4	0				

For dp[1][2] , as A != Y, dp[1][2] = max(dp[0][2], dp[1][1]) Similarly, for other indices of row 1 we can create the table as shown.

02 Step | For dp[2][1] , as X != A , dp[2][1] = max(dp[1][1], dp[2][0])

		A	Y	Z	X
	0	1	2	3	4
0	0	0	0	0	0
A 1	0	1	1	1	1
X 2	0	1	1	1	2
Y 3	0				
T 4	0				

Similarly, for other indices of row 2 , we can create the table as shown.

03 Step | For dp[3][1] , as Y != A , dp[3][1] = max(dp[2][1], dp[3][0])

		A	Y	Z	X
	0	1	2	3	4
0	0	0	0	0	0
A 1	0	1	1	1	1
X 2	0	1	1	1	2
Y 3	0	1	2	2	2
T 4	0				

Similarly, for other indices of row 3 , we can create the table as shown.

04 Step | For dp[4][1] , as T != A , dp[4][1] = max(dp[4][0], dp[3][1])

		A	Y	Z	X
	0	1	2	3	4
0	0	0	0	0	0
A 1	0	1	1	1	1
X 2	0	1	1	1	2
Y 3	0	1	2	2	2
T 4	0	1	2	2	2

Similarly, for other indices of row 4 , we can create the table as shown.

C++ C Java Python C# JavaScript

```cpp
#include <iostream>
#include <vector>
using namespace std;

// Returns length of LCS for s1[0..m-1], s2[0..n-1]
int lcs(string &s1, string &s2) {
    int m = s1.size();
    int n = s2.size();

    // Initializing a matrix of size (m+1)*(n+1)
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    // Building dp[m+1][n+1] in bottom-up fashion
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
```

```
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        // dp[m][n] contains length of LCS for s1[0..m-1]
        // and s2[0..n-1]
        return dp[m][n];
}

int main() {
        string s1 = "AGGTAB";
        string s2 = "GXTXAYB";
        cout << lcs(s1, s2) << endl;

        return 0;
}
```

Output

```
4
```

[Expected Approach 2] Using Bottom-Up DP (Space-Optimization):

One important observation in the above simple implementation is, in each iteration of the outer loop we only need values from all columns of the previous row. So there is no need to store all rows in our DP matrix, we can just store two rows at a time and use them. We can further optimize to use only one array.

> *Please refer this post: A Space Optimized Solution of LCS*

Applications of LCS

LCS is used to implement diff utility (find the difference between two data sources). It is also widely used by revision control systems such as Git for multiple changes made to a revision-controlled collection of files.

Problems based on LCS

- LCS of 3 Strings
- Printing LCS
- Longest Palindromic Subsequence
- Shortest Common Supersequence
- Minimum Insertions and Deletions
- Edit Distance
- Minimum Insertions for Palindrome
- Longest Common Substring
- Longest Palindromic Substring
- Longest Repeated Subsequence
- Count Distinct Subsequences
- Regular Expression Matching

Related Links:

- Print LCS of two Strings
- Dynamic Programming
- LCS and Edit Distance Dynamic
- Programming Practice Problems

Longest Common Subsequence (LCS)

Algorithms

Advanced

Interview Preparation

Practice Problem